

NAME:KRISHITHA .G

REG NO:192524451

Course code:CSA0613

Course title: Design and Analysis of Algorithms

CO2_ASSESSMENT TOOL 3 _ DEBUGGING & OPTIMISATION TASK

Title: Debugging Pair Sum Using Optimized Loop Structure

Aim

To identify and remove redundant comparisons in the brute-force pair sum algorithm, optimize the loop structure, and analyze its time complexity.

Software Used

- Python

Algorithm

- Start.
- Read the array elements and the target sum.
- Traverse the array using the outer loop.
- For each element, use the inner loop starting from the next index (i + 1).
- Check whether the sum of the selected pair equals the target.
- If the target sum is found, display the pair of indices.
- If no pair is found after all comparisons, display "Pair Not Found".
- Stop.

Source Code (Python)

```
main.py    Share  Run
```

```
1 def pair_sum(arr, target):
2     for i in range(len(arr)):
3         for j in range(i + 1, len(arr)): # Avoid duplicate comparisons
4             if arr[i] + arr[j] == target:
5                 return (i, j)
6     return None
7
8 arr = [2, 7, 11, 15]
9 target = 9
10
11 result = pair_sum(arr, target)
12
13 if result:
14     print("Pair found at indices:", result)
15 else:
16     print("Pair not found")
```

OUTPUT

```
Output
```

```
Pair found at indices: (0, 1)
```

```
=== Code Execution Successful ===
```

Analysis

Original Inefficiencies

- The inner loop starts from index 0, causing unnecessary comparisons.
- Duplicate pairs such as (i, j) and (j, i) are checked.
- The algorithm also compares an element with itself (i == j).

Optimized Approach

- Start the inner loop from $i + 1$.
- Each unique pair is checked only once.
- Self-comparisons and duplicate comparisons are eliminated while maintaining correctness.

Documentation

- The optimized algorithm avoids redundant pair comparisons by starting the inner loop from $i + 1$.
- Each unique pair is examined exactly once.
- This reduces unnecessary computations while preserving the correctness of the algorithm.
- Time Complexity: $\Theta(n^2)$
- Space Complexity: $\Theta(1)$

:

